

Tecnológico de Monterrey - Campus Monterrey

Programación de estructuras de datos y algoritmos fundamentales

Evidencia de Pruebas: Evidencia 2: Actividad Integral de Estructuras de Datos Lineales

Juan Ángel Narváez Torres - A00843809

Octubre 19, 2026

Monterrey, Nuevo León

1. Evidencia de ejecución

1.1 Salida del Programa de Pruebas de la Parte A

A continuación se presenta la salida textual obtenida al ejecutar el programa de pruebas de la Parte A, donde se verifican casos de estructura vacía, un solo elemento, operaciones en los extremos en O(1), comportamiento LIFO en Pila, comportamiento FIFO en Fila, recorridos hacia adelante y hacia atrás en la Lista Doblemente Ligada y manejo de excepciones con `try / catch` sin cerrar el programa:

PARTE A: PROGRAMA DE PRUEBAS DE ESTRUCTURAS BASE

```
      PARTE A: PROGRAMA DE PRUEBAS DE ESTRUCTURAS BASE
  Estado inicial (tamaño 0): [Lista vacía]
  Intentando eliminar al inicio en lista vacía:
    [Excepción controlada] Error: La estructura está vacía, no se puede eliminar al inicio.
  Insertando un solo elemento al inicio (10):
    Lista (tamaño 1): 10
  Eliminando el único elemento al inicio:
    Elemento eliminado: 10 | Tamaño actual: 1
  [Lista vacía]
  Insertando en extremos: push_front(20), push_front(10), push_back(30), push_back(40):
    Lista actual (tamaño 4): 10 -> 20 -> 30 -> 40
  Eliminando al inicio (pop_front): 10
    Lista resultante (tamaño 3): 20 -> 30 -> 40
  ¿Pila vacía?: Sí | Tamaño: 0
  Intentando hacer pop() y peek() en Pila vacía:
    [Excepción controlada] Error: No se puede hacer pop sobre una Pila vacía.
    [Excepción controlada] Error: No se puede hacer peek sobre una Pila vacía.
  Caso de un solo elemento: push("Evento_A")
    Tope actual (peek): Evento_A | Tamaño: 1
    Haciendo pop(): Evento_A | ¿Quedó vacía?: No
  Insertando 3 elementos: push("Log_1"), push("Log_2"), push("Log_3"):
    Tamaño de la pila: 3 | Peek: Log_3
    Vaciando pila para comprobar orden LIFO (último en entrar, primero en salir):
      pop() -> Log_3
      pop() -> Log_2
      pop() -> Log_1
  ¿Fila vacía?: Sí | Tamaño: 0
  Intentando hacer dequeue() y front() en Fila vacía:
    [Excepción controlada] Error: No se puede hacer dequeue sobre una Fila vacía.
    [Excepción controlada] Error: No se puede consultar front sobre una Fila vacía.
  Caso de un solo elemento: enqueue("Linea_1")
    Frente actual (front): Linea_1 | Tamaño: 1
    Haciendo dequeue(): Linea_1 | ¿Quedó vacía?: No
  Encolando 3 elementos: enqueue("Primero"), enqueue("Segundo"), enqueue("Tercero"):
    Tamaño de la fila: 3 | Frente: Primero
    Desencolando todo para comprobar orden FIFO (primero en entrar, primero en salir):
      dequeue() -> Primero
      dequeue() -> Segundo
      dequeue() -> Tercero
```

```
Estado inicial (tamaño 0): [Lista doblemente ligada vacía]
Caso un solo elemento: addFirst(100)
  Recorrido hacia adelante: 100
  Recorrido hacia atrás:    100
Eliminando ese único nodo (deleteNode(head)):
  Tamaño actual: 0 | [Lista doblemente ligada vacía]
Insertando al inicio y al final: addFirst(20), addFirst(10), addLast(40), addLast(50)
  Adelante (tamaño 4): 10 <-> 20 <-> 40 <-> 50
Insertando 30 después del segundo nodo (nodo con valor 20) usando insertAfter:
  Recorrido hacia adelante: 10 <-> 20 <-> 30 <-> 40 <-> 50
  Recorrido hacia atrás:    50 <-> 40 <-> 30 <-> 20 <-> 10
Eliminando nodo intermedio (valor 30) y extremos (head=10 y tail=50):
  Adelante (tamaño 2): 20 <-> 40
  Atrás    (tamaño 2): 40 <-> 20
```

1.2 Capturas de pantalla de la Parte B y Casos de Borde

```
Seleccione el archivo que desea cargar:
  1. log607-1.txt
  2. log607-2.txt
  3. Ingresar el nombre del archivo manualmente
Opcion: 1

Procesando buffer de lectura (Fila FIFO)...
Resumen de carga del archivo 'log607-1.txt':
  - Registros cargados exitosamente: 6818
  - L├¡neas con formato invalido omitidas: 0

 MENU PRINCIPAL
1. Ordenar por fecha y hora
2. Buscar logs por dia
3. Mostrar logs por dia en orden descendente
4. Estadisticas del archivo
5. Salir
Seleccione una opcion: 1

Antes de ordenar, ┬┐que crecimiento esperas en el numero de comparaciones?
  1. Aproximadamente n log n
  2. Aproximadamente n^2
Ingresa tu prediccion (1 o 2): 2

 RESULTADOS DEL ORDENAMIENTO
  - Algoritmo usado:                  Merge Sort (in-place sobre lista doble)
  - Archivo procesado:                log607-1.txt
  - Tama▒o de los datos (n):           6818 registros
  - Tiempo de ejecucion:              1.6800 ms
  - Numero de comparaciones realizadas: 78330 (referencia n*log2(n) Γëê 86828)
  - Profundidad m├íxima de recursion:    14 niveles
  - Prediccion ingresada:             n^2
  - ┬┐Coincidio con tu prediccion?:      No, como se uso Merge Sort el crecimiento real fue O(n log n), no n^2.
  - Archivo de salida generado:       output608.out
```

```
 MENU PRINCIPAL
1. Ordenar por fecha y hora
2. Buscar logs por dia
3. Mostrar logs por dia en orden descendente
4. Estadisticas del archivo
5. Salir
Seleccione una opcion: 2

Ingrese el dia a buscar en formato 'Mmm DD AAAA' (ejemplo: Sep 14 2024): Sep 14 2024

 Registros encontrados para Sep 14 2024 (orden ascendente) ---
  Sep 14 2024 00:24:23 10.14.115.26 Social engineering attacks
  Sep 14 2024 00:25:20 10.14.177.24 Suspicious IP Addresses
  Sep 14 2024 00:52:28 10.14.175.238 Security breaches
  Sep 14 2024 01:18:02 10.14.66.138 Anomalous User Activity
  Sep 14 2024 05:14:25 10.14.85.112 Data Exfiltration Attempts
  Sep 14 2024 05:15:49 10.14.58.71 Port Scanning Activity
  Sep 14 2024 11:57:05 10.14.197.208 Cross-Site Scripting
  Sep 14 2024 18:34:13 10.14.65.141 Anomalous User Activity
  Sep 14 2024 20:00:03 10.14.47.235 Data Exfiltration Attempts
  Sep 14 2024 22:23:31 10.14.141.44 Failed Login Attempts
  Sep 14 2024 23:03:07 10.14.175.98 Port Scanning Activity
  Sep 14 2024 23:30:59 10.14.41.52 Suspicious IP Addresses
Total de registros encontrados en Sep 14 2024: 12
```

```
 MENU PRINCIPAL
1. Ordenar por fecha y hora
2. Buscar logs por dia
3. Mostrar logs por dia en orden descendente
4. Estadisticas del archivo
5. Salir
Seleccione una opcion: 3

 Registros del dia Sep 14 2024 en orden DESCENDENTE (usando Pila LIFO) ---
  Sep 14 2024 23:30:59 10.14.41.52 Suspicious IP Addresses
  Sep 14 2024 23:03:07 10.14.175.98 Port Scanning Activity
  Sep 14 2024 22:23:31 10.14.141.44 Failed Login Attempts
  Sep 14 2024 20:00:03 10.14.47.235 Data Exfiltration Attempts
  Sep 14 2024 18:34:13 10.14.65.141 Anomalous User Activity
  Sep 14 2024 11:57:05 10.14.197.208 Cross-Site Scripting
  Sep 14 2024 05:15:49 10.14.58.71 Port Scanning Activity
  Sep 14 2024 05:14:25 10.14.85.112 Data Exfiltration Attempts
  Sep 14 2024 01:18:02 10.14.66.138 Anomalous User Activity
  Sep 14 2024 00:52:28 10.14.175.238 Security breaches
  Sep 14 2024 00:25:20 10.14.177.24 Suspicious IP Addresses
  Sep 14 2024 00:24:23 10.14.115.26 Social engineering attacks
Total mostrados (mas reciente a mas antiguo): 12
```

```
 MENU PRINCIPAL
1. Ordenar por fecha y hora
2. Buscar logs por dia
3. Mostrar logs por dia en orden descendente
4. Estadisticas del archivo
5. Salir
Seleccione una opcion: 4

 ESTADISTICAS DEL ARCHIVO
1. Total de registros validos: 6818

2. Dia(s) con mas incidentes (21 apariciones):
   - Jan 27 2025 (21 incidentes)

3. Tipo(s) de evento mas frecuente(s) (342 apariciones):
   - Malware Detection Alerts (342 apariciones)

4. Direccion(es) IP que aparece(n) mas veces (3 apariciones):
   - 10.14.123.164 (3 apariciones)
   - 10.14.126.198 (3 apariciones)
   - 10.14.161.42 (3 apariciones)
   - 10.14.162.205 (3 apariciones)
   - 10.14.176.221 (3 apariciones)
   - 10.14.186.139 (3 apariciones)
   - 10.14.199.224 (3 apariciones)
   - 10.14.200.76 (3 apariciones)
   - 10.14.242.186 (3 apariciones)
   - 10.14.29.229 (3 apariciones)
   - 10.14.36.7 (3 apariciones)
   - 10.14.69.71 (3 apariciones)
   - 10.14.94.165 (3 apariciones)

 MENU PRINCIPAL
1. Ordenar por fecha y hora
2. Buscar logs por dia
3. Mostrar logs por dia en orden descendente
4. Estadisticas del archivo
5. Salir
Seleccione una opcion: 5

Saliendo de la aplicacion. ┬íHasta luego!
```

2. Resultados del ordenamiento

Archivo	Tamaño (n)	Algoritmo	Predicción	Tiempo de ejecución (ms)	Comparaciones realizadas	Profundidad máxima de recursión
log607-1.txt	6818	Merge Sort	nlogn	1.97 ms aprox	78330	14 niveles
log607-2.txt	6,818	Merge Sort	nlogn	1.21 ms aprox	50624	14 niveles

Interpretación breve de los resultados

Al correr el programa con los 6,818 registros de cada archivo, se nota claramente que Merge Sort cumple con lo prometido de $O(n \log n)$. Para ambos casos la profundidad de recursión se quedó fija en 14 niveles, lo cual tiene todo el sentido porque es justamente el $\log_2$ de la cantidad de datos, así que la pila de llamadas nunca sufre de más ni corre el riesgo de hacer un stack overflow. Lo curioso es que log607-2.txt tardó un poquito menos y usó menos comparaciones (50,624) que el log607-1.txt (78,330); esto pasa porque dependiendo de cómo vengan revueltos los datos al principio, el método de merge a veces avanza más rápido combinando bloques enteros sin hacer tantas comparaciones individuales. Además, gracias a haber puesto bien la condición de menor o igual al juntar los nodos, los logs que tenían exactamente la misma fecha y hora mantuvieron su orden original de llegada sin alterarse.

3. Prompts utilizados y análisis crítico del uso de IA

Durante el desarrollo de la Parte B, utilicé ChatGPT como apoyo puntual para resolver dudas específicas sobre funciones que ya estaba escribiendo y revisar el manejo de punteros. A continuación documento los 3 prompts principales utilizados:

Prompt 1: División de la lista doblemente ligada a la mitad sin usar índices

Prompt utilizado:

"Estoy adaptando el algoritmo Merge Sort que vimos en clase con vectores para que funcione directamente con los punteros NodeD<T>* de mi clase DoublyLinkedList sin copiar los datos a un arreglo. ¿Cómo puedo encontrar el nodo de en medio de una sublista usando dos punteros para partirla en dos sublistas independientes?"

Qué generó y qué modifiqué:

El asistente me explicó la técnica de "tortuga y liebre" (slow avanza un nodo y fast avanza dos nodos en cada iteración). Tomé esa idea para escribir mi método privado split(NodeD<T>* source), pero tuve que adaptarlo a los nombres de mis atributos (next y prev de mi struct NodeD) y agregar la desconexión explícita slow->next = nullptr; y secondHalf->prev = nullptr; para que ambas mitades quedaran bien aisladas antes de las llamadas recursivas.

Prompt 2: Validación limpia de una línea del log con std::stringstream

Prompt utilizado:

"En C++ tengo una línea con formato Sep 29 2024 14:37:38 10.14.19.188 Unauthorized Access Attempts. Ya extraje los primeros 5 campos con stringstream >>, pero la descripción del evento al final tiene espacios. ¿Cómo leo el resto de la línea sin que quede el espacio en blanco inicial ni el salto \r?"

Qué generó y qué modifiqué:

Me sugirió usar std::getline(ss, evt) inmediatamente después de extraer la IP con >> y eliminar el primer carácter si era un espacio. Integré esa lógica dentro de mi método Log::parseLine, y yo agregué por mi cuenta todas las validaciones con throw invalid_argument(...) para verificar meses válidos, días por mes (incluyendo años bisiestos), formato HH:MM:SS y que los 4 octetos de la IP estuvieran entre 0 y 255.

Prompt 3: Error de la IA y corrección

Prompt utilizado:

"Revisa mi función mergeLists(NodeD<T>* first, NodeD<T>* second) para unir dos listas doblemente ligadas ordenadas. Para archivos de más de 6000 registros ¿conviene hacerlo recursivo como en una lista simple o iterativo?"

Error o ineficiencia sugerida por la IA:

Inicialmente la IA me sugirió una función mergeLists **recursiva** (donde cada nodo comparado hacía una llamada recursiva first->next = mergeLists(first->next, second)), y además en su código olvidó actualizar el puntero prev del primer nodo de la lista resultante (mergedHead->prev = nullptr) y el puntero tail de la clase DoublyLinkedList al terminar todo el mergeSort.

Cómo lo detecté y cómo lo corregí:

Detecté dos problemas graves al probarlo:

1. Si mergeLists se hace recursivo para unir dos mitades de tamaño n, realiza hasta n llamadas recursivas anidadas solo para fusionar, lo que dispara la pila de llamadas a $O(n)$ en lugar de mantenerla en $O(\log n)$.
2. Al correr mi prueba de recorrido hacia atrás (printBackward()) después de ordenar, el programa sufría un fallo de segmentación (segmentation fault) o imprimía menos elementos porque el puntero tail de la clase seguía apuntando al nodo que originalmente estaba al final antes de ordenar, no al nuevo último nodo.

Corrección aplicada: Cambié mergeLists por una implementación 100% iterativa usando dos punteros locales (mergedHead y mergedTail) que van enlazando chosen->prev = mergedTail y mergedTail->next = chosen en cada paso sin gastar memoria extra en la pila de llamadas. Además, al final de DoublyLinkedList::mergeSort, agregué un ciclo que asegura que head->prev

= nullptr, verifica todos los enlaces prev y actualiza tail al verdadero último nodo de la lista ya ordenada.

4. Reflexión personal

4.1 Aprendizaje sobre el uso de IA y trabajo propio

En esta actividad comprendí que los asistentes de IA son útiles como herramienta de consulta rápida, pero no sustituyen el entendimiento de cómo funcionan los punteros en memoria. Toda la Parte A (la lista ligada simple con head y tail en $O(1)$, la Pila, la Fila y la Lista Doblemente Ligada) la construí sobre el código que desarrollamos en clase. En la Parte B, haber diseñado yo mismo la estructura de clases me permitió notar de inmediato cuando una sugerencia de la IA rompía la invariante del puntero tail o de los enlaces prev en la lista doblemente ligada. Si no hubiera usado IA en absoluto, probablemente habría tardado un poco más buscando en documentación oficial de C++ (cppreference) los detalles de <chrono> y el manejo de getline tras operator>>, pero la lógica de las estructuras de datos y el ordenamiento habría sido prácticamente la misma.

4.2 Papel de cada estructura en la aplicación: ¿Era necesaria la Pila en la Opción 3?

Cada estructura lineal cumple un rol específico dentro del flujo de datos:

- La Fila (Queue) actúa como un buffer de lectura FIFO (First-In, First-Out): garantiza que las líneas crudas leídas del archivo se procesen exactamente en el mismo orden en que llegaron antes de convertirlas en objetos Log.
- La Lista Doblemente Ligada (DoublyLinkedList) es el almacén central dinámico: permite insertar registros al final en $O(1)$ conforme se validan y re-enlazar nodos en $O(1)$ durante el ordenamiento sin tener que desplazar bloques contiguos de memoria ni redimensionar arreglos.
- Respecto a la Opción 3 (Mostrar logs por día en orden descendente): técnicamente, dado que una DoublyLinkedList ya cuenta con punteros prev en cada nodo (NodeD), no sería estrictamente indispensable usar una Pila si guardáramos un puntero al último registro encontrado de ese día y recorriéramos hacia atrás usando aux = aux->prev hasta salir de la fecha buscada.
    - Ventajas de usar la Pila (Stack): Desacopla por completo el resultado de la búsqueda de la estructura principal. Al hacer push de los registros encontrados en la Opción 2, la Pila almacena únicamente el subconjunto filtrado de ese día y, por su naturaleza LIFO (Last-In, First-Out), al hacer pop entrega automáticamente los eventos del más reciente al más antiguo en $O(1)$ por elemento, funcionando igual incluso si el almacén principal hubiera sido una lista ligada simple sin puntero prev.

- Desventajas de usar la Pila frente a recorrer con prev: La Pila consume memoria adicional $O(k)$ (donde k es la cantidad de registros encontrados en ese día) al crear nuevos nodos en la lista interna de la pila y copiar los objetos Log, mientras que recorrer la lista doblemente ligada hacia atrás con prev tendría un costo de memoria adicional $O(1)$ (sin duplicar nodos).

4.3 ¿Por qué no se puede aplicar búsqueda binaria directamente sobre una lista ligada?

La búsqueda binaria requiere acceder al elemento central de un rango [low, high] en tiempo constante $O(1)$ calculando el índice mid = (low + high) / 2. En un arreglo o std::vector, los elementos están almacenados en posiciones contiguas de memoria, por lo que calcular la dirección de arr[mid] es una simple operación aritmética de punteros ($O(1)$). En cambio, en una lista ligada o doblemente ligada, los nodos están dispersos en el heap y solo se puede llegar al nodo k-ésimo recorriendo enlace por enlace (aux = aux->next) desde head o tail. Por lo tanto, "saltar" al punto medio cuesta $O(n)$ pasos en la primera iteración, $O(n/2)$ en la segunda, $O(n/4)$ en la tercera, y así sucesivamente; la suma converge a $O(n)$, lo que destruye la ventaja logarítmica $O(\log n)$ de la búsqueda binaria y la vuelve igual de lenta que una búsqueda secuencial. Para hacerla posible en $O(\log n)$ de consulta, tendríamos que construir una estructura auxiliar de índice (un arreglo dinámico de punteros NodeD<Log>** de tamaño n donde cada casilla apunte al nodo k-ésimo de la lista ordenada), lo cual tendría un costo de construcción de $O(n)$ en tiempo y $O(n)$ en memoria adicional, y dicho índice tendría que reconstruirse si se insertan o eliminan nodos.

4.4 Interpretación de las mediciones, profundidad de recursión y elección de algoritmo

Al elegir Merge Sort sobre la lista doblemente ligada, el comportamiento fue sumamente consistente tanto en el archivo casi ordenado (log607-1.txt) como en el archivo con orden aleatorio (log607-2.txt). Esto se debe a que el caso promedio, mejor caso y peor caso de Merge Sort tienen complejidad temporal $O(n \log n)$. La profundidad máxima de recursión medida fue de 14 niveles (correspondiente a los 6,818 registros procesados), lo que tiene una relación directa con la pila de llamadas (call stack) del sistema: como en cada nivel el tamaño de la sublista se divide exactamente entre 2, jamás existen más de 14 marcos de activación simultáneos en memoria, evitando cualquier riesgo de Stack Overflow. En contraste, si se hubiera utilizado Quick Sort eligiendo como pivote el primer o el último elemento sobre el archivo log607-1.txt (que ya viene casi ordenado), las particiones habrían quedado completamente desbalanceadas, degradando el tiempo a $O(n^2)$ y haciendo crecer la profundidad de recursión a $O(n)$ niveles en la pila de llamadas. Por ello, Merge Sort por re-enlace de punteros resultó ser la opción ideal y más estable para ordenar listas doblemente ligadas.